

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 07 · LECTURE NOTES

Continuous Control

When you cannot enumerate the actions.

MODULE OUTCOMES

- Explain why value-based methods fail in continuous action spaces.
- Explain deterministic policy gradients and DDPG's structure.
- Explain TD3's three fixes and what each addresses.
- Explain maximum-entropy reinforcement learning and SAC.
- Choose an algorithm for a continuous control problem.

1 The obstruction

THE MAX OVER ACTIONS IS THE WHOLE DIFFICULTY

Q-learning's target contains $\max_a Q(s', a)$ (Module 03 §2). **With a small discrete action set you enumerate it; with a real-valued action vector you cannot.**

And solving that maximisation properly at every step is an optimisation problem per transition — which is CSCE 669's entire subject, executed millions of times inside the inner loop of a training run. **Not impossible, and not affordable.**

There are two ways out. Policy gradients never needed the max at all (Module 05) — a Gaussian policy outputs a mean and a standard deviation, which is a continuous action distribution by construction, and the gradient flows through the log-probability. **So PPO works on continuous control with no modification whatsoever**, which is worth noting before introducing anything new.

Or learn a network that outputs the maximiser, and train that network by gradient ascent through the critic — **which is the deterministic policy gradient, and it is a genuinely neat idea**: the argmax becomes a learned function rather than a computation.

TWO NETWORKS:

a CRITIC $Q(s,a)$, trained by the usual TD error
 an ACTOR $\mu(s)$, trained to OUTPUT the action that
 maximises the critic

THE ACTOR'S UPDATE IS JUST THE CHAIN RULE:

$$\text{grad_theta } Q(s, \mu(s)) = \text{grad_a } Q * \text{grad_theta } \mu$$

The critic is differentiable in the ACTION, so you can back-propagate through it into the actor's parameters. The actor becomes a learned argmax.

-- which also means the actor is only as good as the critic's action-gradient, and A CRITIC THAT IS WRONG IN SOME REGION WILL CONFIDENTLY PUSH THE ACTOR THERE. That loop is the specific failure mode.

DDPG = this, plus a replay buffer and target networks (Module 04), plus exploration noise added to the actor's output -- because a deterministic policy does not explore at all on its own.

AND IT IS NOTORIOUSLY BRITTLE:

very sensitive to hyperparameters
 the critic OVERESTIMATES (Module 03 section 4) and
 the actor exploits exactly the overestimate, which
 is the worst possible interaction
 results vary enormously across random seeds
 (Module 12)

So DDPG is important historically and is NOT what you should use. Sections 2 and 3 are.

2 TD3

Fix	Mechanism	What it addresses
Clipped double Q-learning	Train two critics and use the <i>minimum</i> of the two in the target.	Overestimation (Module 03 §4) — and note this goes beyond unbiasing: it is deliberately pessimistic , because an underestimate is far safer than an overestimate when the actor is going to exploit whichever way the error points.
Delayed policy update	Update the actor once every d critic updates ($d = 2$ typically).	The actor chasing a critic that is still moving — which is Module 06 §2's moving-target problem inside the actor-critic loop.
Target policy smoothing	Add clipped noise to the action used in the target computation.	Sharp spurious peaks in the critic that the actor would exploit — averaging over a small action neighbourhood makes the critic's surface smoother and the peaks less attractive.

All three fixes address the same underlying problem: the actor exploits the critic's errors. So make the critic pessimistic, slow the actor down relative to the critic, and smooth the critic's peaks — three independent attacks on one failure mode. Noticing that is more useful than memorising the three names, and it is a good example of a method whose components look miscellaneous and are not.

3 Soft actor-critic

MAXIMUM-ENTROPY REINFORCEMENT LEARNING

Maximise expected return *plus* policy entropy: $J = E[\sum r_t + \alpha \cdot H(\pi(\cdot|s_t))]$. The entropy term is in the objective, not added to the reward as a bonus.

So the agent is rewarded for keeping its options open, and exploration becomes part of what is being optimised rather than a separate mechanism bolted on beside it (Module 02's ϵ -greedy and noise injection are both bolted on).

Which gives three things at once: sustained exploration without a schedule; **robustness, because a policy that must remain stochastic cannot depend on one narrow precisely-tuned behaviour**; and markedly less hyperparameter sensitivity than DDPG, which is the property practitioners actually notice first.

And α is auto-tuned against a target entropy in modern SAC, by treating the entropy constraint as a dual problem (CSCE 669 Module 06's Lagrangian) — **which removes the one hyperparameter the reframing introduced**, and is a large part of why SAC is pleasant to use in a way its predecessors are not.

	PPO	TD3	SAC
Policy	Stochastic.	Deterministic.	Stochastic, with entropy in the objective.
Data	On-policy — discards batches.	Off-policy — replay buffer.	Off-policy.
Sample efficiency	Low.	High.	High.
Robustness	High.	Medium.	High.
Tuning required	Little.	Some.	Very little.
Best when	The simulator is cheap and massively parallel.	The optimal policy really is deterministic.	The default choice.

SAC is the sensible default for continuous control, and PPO is the default when you have massive parallel simulation and value robustness over sample efficiency — which is the situation in large-scale simulated robotics and in most game applications. **Giving a clear recommendation is defensible here**, which is not true of every choice in this course.

4 Practice

- **Normalise the action space to $[-1, 1]$ and scale to physical units outside the agent. Mismatched action scales are a frequent and entirely silent failure** — an agent whose policy output saturates at a tenth of the usable range trains slowly and badly with nothing to indicate why.
- **Normalise observations with a running mean and variance**, updated online — **the single most valuable implementation detail here as in Module 06 §3**, and for the same conditioning reason.

- **Use a squashed Gaussian policy** — sample from a Gaussian and apply tanh — **and correct the log-probability for the squashing** (the change-of-variables Jacobian). **Forgetting the correction is a real bug** that makes the entropy term wrong and the policy over-deterministic, and it produces no error message.
- **Reward scale matters more than it should. A reward scaled by a thousand changes the magnitude of the critic's targets and therefore the effective learning rate** — so normalise or scale rewards deliberately rather than inheriting whatever the environment emits.
- **And if your target is a real physical system, read Module 08 and Module 09 before starting**, because the sample counts in Module 01 §2 make direct training infeasible and the alternatives have their own requirements.

SIMULATION TO REALITY

Training on a real robot is usually infeasible at the sample counts of Module 01 §2 — millions of steps at one step per second is weeks of continuous operation, with wear and safety costs — **so you train in simulation and transfer**.

And the simulation is wrong. Friction, contact dynamics, actuator latency, sensor noise, and compliance are all approximated, and **a policy trained on the simulator exploits the inaccuracies** — it finds the contact configuration the physics engine handles badly and builds its behaviour on it. **This is CSCE 669 Module 12 §4 and Module 01 §3's model-exploitation problem, in its most expensive form.**

Domain randomisation is the standard answer: randomise the simulator's physical parameters across every episode so that **the policy must work across a range of dynamics, with the real system's parameters somewhere inside that range**. A policy that cannot rely on a specific friction coefficient cannot exploit a wrong one.

It works, imperfectly, and it costs sample efficiency — the policy becomes conservative, because it must be robust to dynamics it is not currently experiencing. **And report the real-system performance rather than the simulated one**, which is **the only number that means anything** and is the one most frequently omitted.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Haarnoja et al. — Soft Actor-Critic (free)	§3's method , with the maximum-entropy framing derived and the auto-tuned temperature in the follow-up paper.
Fujimoto et al. — Addressing Function Approximation Error in Actor-Critic Methods (TD3) (free)	§2's three fixes , with the overestimation analysis that motivates all three.

Resource	What it covers for this module
Lillicrap et al. — Continuous control with deep reinforcement learning (DDPG) (free)	§1's deterministic policy gradient — read for the idea, not as a recommendation.
Tobin et al. — Domain Randomization (free)	§4's transfer method , and a clear statement of the reality-gap problem it addresses.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Try to run DQN on a continuous action space** and articulate exactly where it fails.
- 2 **Run PPO on a continuous control task** with no modification, and confirm it works.
- 3 **Implement DDPG** and run five seeds. Report the spread.
- 4 **Plot the critic's Q-values over training** and look for overestimation.
- 5 **Add clipped double Q** and report the change in Q-values and performance.
- 6 **Add each of TD3's other two fixes** separately and report each effect.
- 7 **Implement SAC** and compare sample efficiency against PPO on the same task.
- 8 **Omit the tanh log-probability correction** and report what happens to the entropy.
- 9 **Scale the reward by 1000** and report the effect on learning.
- 10 **Randomise a simulator parameter** and compare the policy's robustness with and without.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does the max over actions obstruct value-based methods?
- 2 Name two ways around it.
- 3 Explain the deterministic policy gradient and its chain rule.
- 4 What is DDPG's specific failure loop?
- 5 Name TD3's three fixes and the single problem they all address.
- 6 State the maximum-entropy objective and the three things it buys.
- 7 Why is α auto-tuned, and by what mechanism?
- 8 Compare PPO, TD3, and SAC on six axes and give a default.
- 9 Name two silent bugs specific to continuous control.

10 What is domain randomisation, what does it cost, and what should you report?